



安徽大學
Anhui University

《JAVA程序设计》

第三章 表达式和流程控制语句

🎤 主讲人：黄小平

本章目录



安徽大學
Anhui University

1

表达式

2

流程控制

3

简单的输入输出

4

本章习题解答



3.1 表达式



安徽大學
Anhui University

表达式由**运算符**和**操作数**构成，对操作数进行运算符指定的操作，并得到一个结果。

- **Java运算符**按功能可分为：算术运算符、关系运算符、逻辑运算符、位运算符、赋值运算符、条件运算符、数组下标运算符等。操作数可以是变量、常量或方法调用等；
- 如果表达式中仅含有算术运算符，如 “*”，则为算术表达式，计算结果是算术量（“+”用于字符串连接除外）；
- 如果表达式中含有关系运算符，如 “>”，则为关系表达式，计算结果是逻辑值，即true或false；
- 如果表达式中含有逻辑运算符，则为逻辑表达式，相应的计算结果为逻辑值；

3.1.1 操作数



安徽大學
Anhui University

1. 常量

□ 常量操作数很简单，只有简单数据类型和String类型才有相应的常量形式；

常量

23.59

-1247.1f

true

"This is a String"

含义

double型常量

float型常量

boolean型常量

String型常量

3.1.1 操作数



安徽大學
Anhui University

2. 变量

- 变量是存储数据的基本单元，它可以用作表达式中的操作数，在使用之前要先说明；
- 变量说明的基本格式为：

类型 变量名1 [= 初值1][,变量名2 [= 初值2]]...;

3.1.1 操作数



安徽大學
Anhui University

类型 变量名1 [= 初值1][,变量名2 [= 初值2]]...;

- 类型既可以是简单类型，也可以是类类型。有时也把类类型的变量称为引用。
- 变量说明的地方有两处，一处是在方法内，另一处是在类定义内。方法内定义的变量称作**自动变量**，局部变量、临时变量或栈变量。类中定义的变量就是它的**成员变量**。
- 简单类型的变量在说明之后，系统自动在内存分配相应的存储空间。说明引用后，系统只分配引用空间，程序员要调用new来创建对象实例，然后才分配相应的存储空间。

3.1.1 操作数



3. 变量初始化

- ❑ Java不允许将未经初始化的变量用做操作数。对简单变量在说明的同时可以进行初始化;
- ❑ 创建一个对象后, 使用**new**运算符分配存储空间时, 系统按表中的值自动初始化成员变量;

类型	初始值
byte	(byte) 0
short	(short) 0
int	0
long	0L
float	0.0f
double	0
char	'\u0000' (NULL)
boolean	FALSE
所有引用类型	null

3.1.1 操作数



安徽大學
Anhui University

3. 变量初始化

- 自动变量在使用之前必须初始化;
- 编译器扫描代码, 判定每个变量在首次使用前是否已被**显式初始化**。如果某个变量没有初始化, 会发生编译时错误;

□ 例3-2

```
int x = (int)( Math.random() * 100 );  
int y;  
int z;  
if (x > 50) {  
    y = 9;  
}
```

z = y + x; // 可能在初始化之前使用, 导致编译错误

3.1.1 操作数



安徽大學
Anhui University

3. 变量作用域

- ❑ 变量的作用域是指可访问该变量的代码范围;
 - ✓ 类中定义的成员变量的作用域是整个类;
 - ✓ 方法中定义的局部变量的作用域是从该变量的说明处开始到包含该说明的语句块结束处;
- ❑ 块内说明的变量将屏蔽其所在类定义的同名变量; 在同一块内如果定义两个同名变量则引起冲突。
- ❑ Java允许屏蔽, 单冲突会引起编译器错误。

3.1.1 操作数



安徽大學
Anhui University

程序3-1

```
1 public class Customer {
2
3
4     public static void main(String[] args) {
5         Customer customer = new Customer();
6         String name = "John Smith";
7         {
8             // 下列说明是非法的
9             String name = "Tom David";
10            customer.name = name;
11            System.out.println(
12                "The customer's name: " + customer.name);
13        }
14    }
15    private String name;
16 }
```

3.1.1 操作数



安徽大學
Anhui University

程序3-3

```
class Customer {  
  
    public static void main(String[] args) {  
        Customer customer = new Customer();  
        {  
            String name = "Tom David";    //第6行  
            customer.name = name;  
            System.out.println("The customer's name: " + customer.name);  
        }  
        // 下面的再说明是正确的  
        String name = "John Smith";        //第11行  
        customer.name = name;  
        System.out.println(  
            "The customer's name: " + customer.name);  
    }  
    private String name;  
}
```

3.1.2 运算符



安徽大學
Anhui University

1. 算术运算符

- ❑ 算术运算符包括通常的加 (+)、减 (-)、乘 (*)、除 (/)、取模 (%), 完成整型或浮点型数据的算术运算;
- ❑ 算术运算符还有 “++”、“--” 两种, 分别表示加1和减1操作;

3/2	// 结果是1
15.2 % 5	// 结果是0.2
5%-3	// 结果是2
-5%-3	// 结果是-2

3.1.2 运算符



安徽大學
Anhui University

□ 思考题：执行以下语句之后，i和j各为多少？

```
int i=3, j;  
j=(i++)+(i++)+(++i);  
System.out.println(i);  
System.out.println(j);
```

```
i=6;  
j=(i++)+(i++)+(++i);  
    3      4      6
```

3.1.2 运算符



安徽大學
Anhui University

2. 关系运算符

- ❑ 关系运算符操作得到的**结果**是true或者false;
- ❑ 关系运算符用来比较两个值，包括大于（>）、大于等于（>=）、小于（<）、小于等于（<=）、等于（==）、不等于（!=）6种；
- ❑ 关系运算符都是二元运算符，运算的结果是一个逻辑值；
- ❑ Java允许 “==” 和 “!=” 两种运算用于任何数据类型；
 - ✓ 可以判定两个实例是否相等；

3.1.2 运算符



安徽大學
Anhui University

3. 逻辑运算符

- 逻辑运算符操作得到的**结果**是true或者false;
- 逻辑运算符包括逻辑与（&&）、逻辑或（||）和逻辑非（! ）。前两个是二元运算符，后一个是一元运算符
- Java对逻辑与和逻辑或提供短路操作功能。进行运算时，先计算运算符左侧表达式的值，如果使用该值能得到整个表达式的值，则跳过运算符右侧表达式的计算，否则计算运算符右侧表达式，并得到整个表达式的值。

3.1.2 运算符



安徽大学
Anhui University

3. 逻辑运算符

```
String unset = null;  
if ((unset != null) && (unset.length() > 5)) {  
    // 对unset进行某种操作  
}
```

空串**unset**不能使用，因此不能访问**unset.length()**

if()语句中的逻辑表达式是合法的，且完全安全。因为第一个子表达式（**unset != null**）结果为假，它马上导致整个表达式的结果为假。所以**&&**运算符跳过不必要的（**unset.length() > 5**）计算，因为没有计算它，避免了空指针异常。

3.1.2 运算符



安徽大學
Anhui University

3. 逻辑云端的“短路特性” &&, ||, 都具备短路特性

运行下面代码，x,y,z输出是多少？

```
int x,y,z;  
bool b;  
x=y=z=0;  
b=(++x || ++y) && ++z;
```

3.1.2 运算符



安徽大學
Anhui University

4. 位运算符

- 位运算符用来对二进制位进行操作，包括按位取反（~）、按位与（&）、按位或（|）、异或（^）、右移（>>）、左移（<<）及无符号右移（>>>）。**位运算符只能对整型和字符型数据进行操作。**
- 运算符“>>”执行算术右移，它使用最高位填充移位后左侧的空位。右移的结果为：每移一位，第一个操作数被2除一次，移动的次数由第二个操作数确定。

```
128 >> 1    得到 64
256 >> 4    得到 16
-256 >> 4   得到 -16
```

3.1.2 运算符



安徽大學
Anhui University

4. 位运算符

- ❑ 逻辑右移或叫无符号右移运算符 $\gg$ 只对位进行操作，而没有算术含义，它用0填充左侧的空位；
- ❑ 算术右移不改变原数的符号，而逻辑右移不能保证这一点；

<code>(byte)0x80 >> 2</code>	得到 -32
<code>0xa2 >>> 2</code>	得到 40
<code>(byte) 0xa2 >> 2</code>	得到 -24
<code>(byte) 0xa2>>>2</code>	得到 1073741800

3.1.2 运算符



安徽大學
Anhui University

4. 位运算符

- 移位运算符约简其右侧的操作数，当左侧操作数是int类型时，右侧以32取模；当左侧是long类型时，右侧以64取模；

```
int x;
```

```
x = x >>> 32;
```

x的结果不改变，而不是通常期望的0。

这样可以保证不会将左侧操作数完全移走。

- “>>>” 运算符只用于整型，它只对int或long值起作用。如果用于short或byte值，则在进行“>>>”操作之前，使用符号扩展将其提升为int型，然后再移位；

3.1.2 运算符



安徽大學
Anhui University

5. 其他运算符

- Java中的运算符还包括扩展赋值运算符（=、+=、-=、*=、/=、%=、&=、|=、^=、>>=、<<=）及（>>>=），条件运算符（?:），点运算符（.），实例运算符（instanceof），（new）运算符，数组下标运算符（[]）等。
- 扩展赋值运算符是在赋值号（=）前再加上其他运算符，是对表达式的一种简写形式。如果有赋值语句：

var = var op expression;

- 其中，var是变量，op是算术运算符或位运算符，expression为表达式。使用扩展赋值运算符可表示为：**var op= expression;**

3.1.2 运算符



安徽大學
Anhui University

5. 其他运算符

□ 条件运算符 (?:) 是三元运算符，它的一般形式为：

表达式 ? 语句1 : 语句2;

□ 表达式得到一个逻辑值，根据该值的真假决定执行什么操作。如果值为真，执行语句1，否则执行语句2。

□ 注意，语句1和语句2需要返回相同的类型，且不能是void。

□ 举例：

$x = a > b ? a=15 : b=0;$

3.1.2 运算符



6. 运算符的优先级

- 在对一个表达式进行计算时，如果表达式中含有多种运算符，则要按运算符的优先顺序依次从高向低进行，同级运算符则从左向右进行。括号可以改变运算次序；

	., [], ()
	++, --, !, ~, instanceof
	new (type)
高	*, /, %
	+, -
↓	>>, >>>, <<
	<, >, <=, >=
低	==, !=
	&
	^
	&&
	?:
	=, +=, -=, *=, /=, %=
	&=, =, <<=, >>=, >>>=

3.1.2 运算符



安徽大學
Anhui University

6. 运算符的优先级

□ 单目乘除为关系，逻辑三目后赋值。

单目：单目运算符+ -(负数) ++ -- 等

乘除：算数单目运算符* / % + -

为：位移单目运算符<< >>

关系：关系单目运算符> < >= <= == !=

逻辑：逻辑单目运算符&& || & | ^

三目：三目单目运算符A > B ? X : Y

后：无意义，仅仅为了凑字数

赋值：赋值=

```
--y || ++x && ++z;
```

```
int m = 12;  
int n = m << 1 + 2;  
int n = m << (1 + 2); //这样更直观
```

2. 若有 a=3,则执行下列语句为a的值为? a+=a*=a;

A. 3 B. 12 C. 9 D.18

```
int a = 5, b=4;  
int c = a++- --b*++a/b-- >>2%a--;  
a=?, b=?, c=?
```

3.1.3 表达式的提升和转换



- ❑ Java语言不支持变量类型间的自动任意转换，有时必须显式地进行变量类型的转换。一般的原则是，变量和表达式可转换为更一般的形式，而不能转换为更受限制的形式；
 - ✓ 例如，int型表达式可看作是long型的；而long型表达式当不使用显式转换时是不能看作int型的；
 - ✓ 一般地，如果变量类型至少与表达式类型一样（即位数一样多），就可以认为表达式是赋值相容的；

3.1.3 表达式的提升和转换



- 例3-7

- `long bigval = 6;` // 6是整型量，所以该语句**正确**
- `int smallval = 99L;` // 99L是长整型量，该语句**错误**
- `float z = 12.414F;` // 12.414F是浮点量，该语句**正确**
- `float z1 = 12.414;` // 12.414是双精度量，该语句**错误**

99L是长整型量，smallval是int型量，赋值不相容。同样，12.414是双精度型的，不能赋给单精度变量z1

3.1.3 表达式的提升和转换



- 当表达式不是赋值相容时，有时需进行转换以便让编译器认可该赋值
 - ✓ 让一个long型值“挤”入int型变量中。显式转换如下：

```
long bigValue = 99L;  
int squashed = (int) (bigValue);
```

- 转换时，目标类型用括号括起来，放到要修改的表达式的前面。为避免歧义，被转换的整个表达式最好也用括号括起来；

3.1.4 数学函数



安徽大學
Anhui University

□ 当数学函数类Math，几个常用的函数调用：

■ Math.sin(0)	//返回0.0，这是double类型的值
■ Math.cos(0)	//返回1.0
■ Math.tan(0.5)	//返回0.5463024898437905
■ Math.round(6.6)	//返回7
■ Math.round(6.3)	//返回6
■ Math.ceil(9.2)	//返回10 .0
■ Math.ceil(-9.8)	//返回-9 .0
■ Math.floor(9.2)	//返回9 .0
■ Math.floor(-9.8)	//返回-10 .0
■ Math.sqrt(144)	//返回12.0
■ Math.pow(5,2)	//返回25.0
■ Math.exp(2)	//返回7.38905609893065
■ Math.log(7.38905609893065)	//返回2.0
■ Math.max(560, 289)	//返回560
■ Math.min(560, 289)	//返回289
■ Math.random()	//返回0.0到1.0之间双精度的一个随机数值

3.2 流控制



安徽大學
Anhui University

□ 语句与表达式

- ✓ 每个表达式都会得到一个值，即表达式的计算结果。语句也会有一个值，但这个值并不是语句的计算结果，而是执行结果

□ 语句是Java的最小执行单位，语句间以分号 (;) 作为分隔符。语句分为简单语句及复合语句；

- ✓ **简单语句**就是通常意义下的一条语句，即单语句；
- ✓ **复合语句**是一对花括号 “{” 和 “}” 括起来的语句组，也称为 “块”，块后没有分号；

3.2.1 表达式语句



□ 表达式可当作一个值，有的表达式也可当作语句；

- `customer1 = new Customer();`
- `point2 = new Point();`
- `x = y+12;`
- `x++;`

□ 方法调用通常返回一个值，一般用在表达式中。有的方法调用可直接当作语句。

- `y=get_sin(x);`
- `System.out.println("Hello World!");`

3.2.2 块



□ 块是一对花括号 “{” 和 “}” 括起来的语句组。

例如：

```
{ }
```

```
{
```

```
    Point point1 = new Point();  
    int x = point1.x;
```

```
}
```

□ 方法体是一个块。块还用在流控制的语句中，如if语句、switch语句及循环语句；

3.2.3 分支语句



- ❑ 分支语句共有两种：**if** 语句和 **switch** 语句；
- ❑ **if** 语句：
 - 是单重选择，最多只有两个分支。
 - if语句的基本格式是：
- ❑ if关键字之后的逻辑表达式必须得到一个逻辑值，不能象其他语言那样以数值来代替。因为Java不提供数值与逻辑值之间的转换

```
if (逻辑表达式)
    语句1;
[else
    语句2;
]
```

```
int x = 3;
if (x)
{...}
```

3.2.3 分支语句



安徽大學
Anhui University

□ if 语句常见的形式:

形式一

```
if (逻辑表达式) {  
    //逻辑表达式为true时要执行的语句;  
}
```

形式二

```
if (逻辑表达式) {  
    //逻辑表达式为true时要执行的  
    语句;  
}  
else {  
    //逻辑表达式为false时要执行的  
    语句;  
}
```

形式三: if语句的嵌套

```
if (逻辑表达式1) {  
    //逻辑表达式1为true时要执行的语句;  
}  
else if (逻辑表达式2) {  
    //逻辑表达式1为false, 但逻辑表达式2  
    为true时要执行的语句;  
}  
...  
else {  
    //前面的逻辑表达式全为false时要执行  
    的语句;  
}
```

3.2.3 分支语句



安徽大學
Anhui University

□ 例3-8

```
1  int count;  
2  count = getCount();  
3  if (count < 0) {  
4      System.out.println("Error: count value is negative!");  
5  }  
6  else {  
7      System.out.println("There will be " + count +  
8      " people for lunch today.");  
9  }
```

3.2.3 分支语句



□ else子句属于逻辑上离它最近的if语句

```
if (firstVal == 0)
    if (secondVal == 1)
        firstVal++;
    else
        firstVal--;
```

这两个匹配

```
1  if (firstVal == 0){
2      if (secondVal == 1)
3          firstVal++;
4  }
5  else
6      firstVal--;
```

这两个匹配

3.2.3 分支语句



□ switch 语句

- switch语句含义与嵌套的if语句是类似的，格式更加简捷;

```
switch (表达式) {  
    case c1:  
        语句组1;  
        break;  
  
    .....  
    case ck:  
        语句组k;  
        break;  
    [default:  
        语句组;  
        break;]  
}
```

3.2.3 分支语句



□ **switch** 语句: 表达式的计算结果必须是int型或字符型, 即是int型赋值相容的。当用byte、short或char类型时, 要进行提升。switch语句不允许使用浮点型或long型表达式。c1、c2、...、ck是int型或字符型常量。default子句是可选的, 并且, 最后一个break语句完全可以不写。

例3-11:

```
1 // colorNum是整型变量
2 switch (colorNum) {
3     case 0:
4         setBackground(Color.red);
5         break;
6     case 1:
7         setBackground(Color.green);
8         break;
9     default:
10        setBackground(Color.black);
11        break;
12 }
```

3.2.3 分支语句



安徽大學
Anhui University

□ **switch** 语句练习:

- 根据月份，输出具体某个月的天数。
- 用Java语言编写，实现以上功能。

3.2.3 分支语句



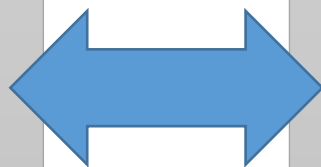
安徽大學
Anhui University

□ **switch**语句和**if**语句可以互相代替

例3-11:

使用switch语句:

```
static int daysInMonth(int month) {  
    int days;  
    switch(month) {  
        case 2: days = 28; break;  
        case 4:  
        case 6:  
        case 9:  
        case 11: days = 30; break;  
        default: days = 31;  
    }  
    return(days);  
}
```



使用if语句:

```
static int daysInMonth(int month) {  
    if (month == 2)  
        return(28);  
    if ((month == 4) || (month == 6) ||  
        (month == 9) || (month == 11))  
        return(30);  
    return(31);  
}
```

3.2.4 循环语句



安徽大學
Anhui University

□ 三种循环语句：**for** 语句、**while**语句和**do**语句

□ **for** 语句的语法格式：

```
for (初始语句; 逻辑表达式; 迭代语句){  
    语句;  
}
```

□ 先执行初始语句，判断逻辑表达式的值，当逻辑表达式为真时，执行循环体语句，执行迭代语句，然后再去判别逻辑表达式的值。直到逻辑表达式的值为假时，循环结束

3.2.4 循环语句



安徽大學
Anhui University

□ for 语句示例

```
for (int i = 0; i < 3; i++) {  
    System.out.println("Are you finished yet?");  
}  
System.out.println("Finally!");
```



Are you finished yet?
Are you finished yet?
Are you finished yet?
Finally!

- 该段程序共执行3次第2行的输出语句（i为0，1，2时）；当i等于3时，逻辑表达式的值为假，退出循环，执行第4行语句

3.2.4 循环语句



安徽大學
Anhui University

□ for 语句：死循环

```
for ( ; ; )  
    System.out.println("Always print!");
```



```
for ( ; true ; )  
    System.out.println("Always print!");
```

□ 初始语句及迭代语句包含多个语句时的情况：

例3-16

```
int sumi = 0, sumj = 0;  
for ( int i = 0, j = 0; j < 10; i++, j++) {  
    sumi += i;  
    sumj += j;  
}
```

3.2.4 循环语句



安徽大學
Anhui University

□ while 循环

- for语句中常常用循环控制变量显式控制循环的执行次数。当程序中不能明确地指明循环的执行次数时，可以仅用逻辑表达式来决定循环的执行与否。这样的循环可用while语句来实现
- while循环的语法是：

while (逻辑表达式)
语句;

- 和if语句一样，while语句中的逻辑表达式亦不能用数值来代替

3.2.4 循环语句



安徽大學
Anhui University

□ while语句的语义：

- 计算逻辑表达式，当逻辑表达式为真时，重复执行循环体语句，直到逻辑表达式为假时结束。如果第一次检查时逻辑表达式为假，则循环体语句一次也不执行。如果逻辑表达式始终为真，则循环不会终止
- 例3-14的for语句可以改写为例3-17中的while语句

```
int i = 0;
while (i < 3) {
    System.out.println("Are you finished yet?");
    i++;
}
System.out.println("Finally!");
```

3.2.4 循环语句



□ do-while循环语句：

- do语句与while语句很相似。它把while语句中的逻辑表达式移到循环体之后。do语句的语法格式是：

```
do  
    语句;  
while (逻辑表达式);
```

□ do 语句的语义是：

- 首先执行循环体语句，然后判定逻辑表达式的值，当表达式为真时，重复执行循环体语句，直到表达式为假时结束。不论逻辑表达式的值是真是假，do循环中的循环体都至少执行一次

3.2.4 循环语句



安徽大學
Anhui University

□ do-while实例

```
1  do
2  int i = 0;
3  do {
4      System.out.println("Are you finished yet?");
5      i++;
6  } while (i < 3);
7  System.out.println("Finally!");
```

3.2.4 循环语句



安徽大學
Anhui University

□ **for**、**while**和**do-while**语句之间可以互相替换

do

语句1;

while (逻辑表达式);

等价于:

语句1;

while(逻辑表达式)

语句1;

3.2.5 break与continue语句



安徽大學
Anhui University

□ Java语言抛弃了goto语句，代之以两条特殊的break和continue语句；

□ 标号

- 标号可以放在for、while或do语句之前，其语法格式为：

标号:语句;

- 标号使用例子

```
out: for (int i=0; i<10; i++) {  
    while (x<50) {  
        if ( i*x == 400)  
            break out;  
    }  
}
```

3.2.5 break与continue语句



安徽大學
Anhui University

□ break语句

- 可用于三类语句中：一类是switch语句中，一类是for、while及do等循环语句中，还有一类是块语句中
- 在switch语句及循环语句中break的语义是跳过本块中余下的所有语句，转到块尾，执行其后的语句

```
for (int i = 0; i < 100; i++) {  
    if ( i == 5 )  
        break;  
    System.out.println("i= " + i);  
}
```

3.2.5 break与continue语句



□ break语句

- break语句的第三种使用方法是和标号结合使用，其语义是跳出标号所标记的语句块，继续执行其后的语句。多用于嵌套块中，控制从内层块跳到外层块之后。其语法格式为：

break 标号;

```
out: for (int i=0; i<10; i++) {  
    while (x<50) {  
        if ( i*x == 400)  
            break out;  
    }  
}
```

3.2.5 break与continue语句



安徽大學
Anhui University

□ continue语句

- 在循环语句中，**continue**可以立即结束当次循环而执行下一次循环，当然执行前先判断循环条件是否满足；
- **continue**语句也可以和标号一起使用，立即结束标号标记的那重循环的当次执行，开始下一次循环。多用于多重循环中。其语法格式为：

continue 标号;

3.2.5 break与continue语句



安徽大學
Anhui University

□ continue语句

```
outer: for (int i = 0; i < 10; i++) {  
    for (int j = 0; j < 20; j++){  
        if ( j>i ) {  
            System.out.println();  
            continue outer;  
        }  
        System.out.print("* ");  
    }  
}
```

```
*  
* *  
* * *  
* * * *  
* * * * *  
* * * * * *  
* * * * * * *  
* * * * * * * *  
* * * * * * * * *  
* * * * * * * * *
```

3.2.6 注释语句



安徽大學
Anhui University

□ Java中共有三种注释形式：

- `//` 在一行内的注释
- `/*` 一行或多行的注释 `*/`
- `/**` 文档注释 `*/`

- ◆ 第一种形式表示从 `“//”` 开始一直到行尾均为注释，一般用它对说明的变量、一程序的作用作简短说明。
- ◆ 第二种形式可用于多行注释，`“/*”` 是注释的开始，`“*/”` 表示注释结束，`“/*”` 和 `“*/”` 之间的所有行均是注释语句。这种注释多用来说明方法的功能等。
- ◆ 第三种形式是文档注释。文档注释放在（一个变量或是一个函数的）说明之前，表示该段注释应包含在自动生成的任何文档中(即由javadoc命令生成的HTML文件)。

3.3 简单的输入输出



- ❑ 输入输出语句，是计算机程序实现人机交互的关键环节。Java提供了几个供输入输出的基本类；
- ❑ Scanner类
 - Scanner类属于java.util包。它提供了许多方法，可用来方便地读入不同类型的输入值。
 - 要调用Scanner类的方法，必须先创建一个对象。
 - 下面的说明创建了一个Scanner类对象，它读入键盘输入：

```
Scanner scan = new Scanner(System.in);
```

3.3 简单的输入输出



安徽大學
Anhui University

□ Scanner类

```
Scanner scan = new Scanner(System.in);
```

- Scanner类的构造方法接受一个参数，这个参数代表输入源。System.in对象代表标准输入流，默认是键盘。
- next()方法用于读取下一个输入对象；
- nextLine()方法读入当前行的所有输入，直到行尾。
- nextInt()和nextDouble()用来读入不同类型的数据。

3.3 简单的输入输出



Scanner类的应用实例

```
■ // 定义变量
■ int age=0;
■ double weight,height,bmi;
■ // 创建Scanner输入对象
■ Scanner scanner=new Scanner(System.in);
■ System.out.println("请输入您的年龄: ");
■ age=scanner.nextInt();
■ System.out.println("请输入您的体重(kg): ");
■ weight=scanner.nextDouble();
■ System.out.println("你的身高是(m):");
■ height=scanner.nextDouble();
■ // 计算BMI
■ bmi=weight/(height*height);
■ System.out.println("你的BMI:"+bmi);
```

3.3 简单的输入输出



□ NumberFormat类和DecimalFormat类

- 用于格式化输出
- NumberFormat类提供对数值进行格式化操作的一般功能。不能使用new运算符实例化一个NumberFormat对象。
- DecimalFormat类按惯例使用new运算符来实例化对象。

```
■ // 计算BMI
■ bmi=weight/(height*height);
■ System.out.println("你的BMI:"+bmi);
■ // 格式化数据
■ DecimalFormat fmt=new DecimalFormat("0.###");
■ System.out.println("你的BMI:"+fmt.format(bmi));
```

3.4 猜数字大小游戏



安徽大學
Anhui University

- 利用输入输出语句，while或者do-while语句，实现猜数字大小。
 - 数字大小调用随机数10-100之间的整数
 - 情景提示：猜大了，猜小了，恭喜你猜对了

3.4 猜数字大小游戏



安徽大學
Anhui University

```
public static void main(String[] args) {  
    // 1. 调用随机数, 产生一个10-90的随机数  
    double data =Math.random()*90+10;  
    long n=Math.round(data);  
    // 2. 初始化输入  
    Scanner scanner=new Scanner(System.in);  
    long guess=0;  
    System.out.println("系统已经产生了一个10-90之间的随机数, 请猜: ");  
    do {  
        guess=scanner.nextInt();  
        if (guess>n) {  
            System.out.println("猜大了! 请继续输入: ");  
        }  
        else if (guess<n) {  
            System.out.println("猜小了! 请继续输入: ");  
        }  
    }while(guess!=n);  
    // 游戏结束  
    System.out.println("恭喜你, 猜对了!");  
}
```

本章讲课结束

谢 谢！